



Bahasa C++: Function Template dan Kelas Generik

IF2210 – Semester II 2025/2026

Sumber: Diktat Bahasa C++ oleh Hans Dulimarta

Modernisasi oleh Satrio Adi Rukmono

Template = Abstraksi pada Level Tipe

- Template memungkinkan kita mendefinisikan algoritma atau struktur data yang sama untuk berbagai tipe, selama tipe tersebut mendukung operasi yang digunakan.
- Template bekerja pada compile-time (*type parameterization*), bukan melalui hierarki kelas.
- Ini membedakan dari *polymorphism*.

Function Template

Latar belakang

- Seringkali kita membutuhkan suatu operasi yang sama secara konseptual, tetapi dapat bekerja untuk berbagai tipe yang memenuhi operasi tertentu.
- Contoh: fungsi `min()` dapat diterapkan untuk `int` maupun `float`.

```
int min(int a, int b) { return a < b ? a : b; }  
float min(float a, float b) { return a < b ? a : b; }
```

- Untuk setiap tipe yang akan dimanipulasi oleh fungsi `min()`, harus ada sebuah fungsi untuk tipe tersebut.
 - Implementasinya identik, hanya tipenya yang berbeda.

Bagaimana mengatasinya?

- Trik yang biasa digunakan dalam C adalah dengan definisi makro:

```
#define mmin(a,b) ((a) < (b) ? (a) : (b))
```

- Namun makro tersebut dapat memberikan efek yang tidak diinginkan pada contoh *statement* berikut:

```
if (mmin(x++, y) == 0) printf ("...");
```

yang akan diekspansi sebagai:

```
if (((x++) < (y) ? (x++) : (y)) == 0) printf ("...");
```

- Substitusi makro $\neq$ pemanggilan fungsi.

Solusi: function template

- Deklarasi: menggunakan *prefix* “template <class XYZ>” sebelum nama fungsi.
Contoh:

```
template <class T>
T min(T a, T b) {
    return a < b ? a : b;
}
```

- Dipanggil dengan cara:

```
int a, b, c;
c = min<int> (a, b);
float x, y, z;
z = min<float> (x, y);
```

Catatan-catatan (1)

- Banyaknya nama tipe (kelas) yang dicantumkan di antara ‘<’ dan ‘>’ dapat lebih dari satu. Setiap nama tipe harus didahului oleh kata kunci `class`.

Contoh:

```
template <class T1, class T2>
```

- Nama tipe yang dicantumkan di antara ‘<’ dan ‘>’ harus tercantum sebagai *function signature*. Contoh:

```
template <class T1, class T2, class T3>
```

```
T1 myFunc(T2 a, T3 b) {...} // error: T1 bukan bagian dari signature
```

Catatan-catatan (2)

- Definisi template fungsi dapat disertai oleh definisi "non-template" dari fungsi tersebut.

```
template <class T>  
T min_arr(const T x[], int size) {...} // template
```

```
Complex min_arr(const Complex x[], int size) {...} // non-template
```

- Apa yang akan terjadi pada pemanggilan berikut?

```
Complex c[5], d;  
d = min_arr(c, 5);
```

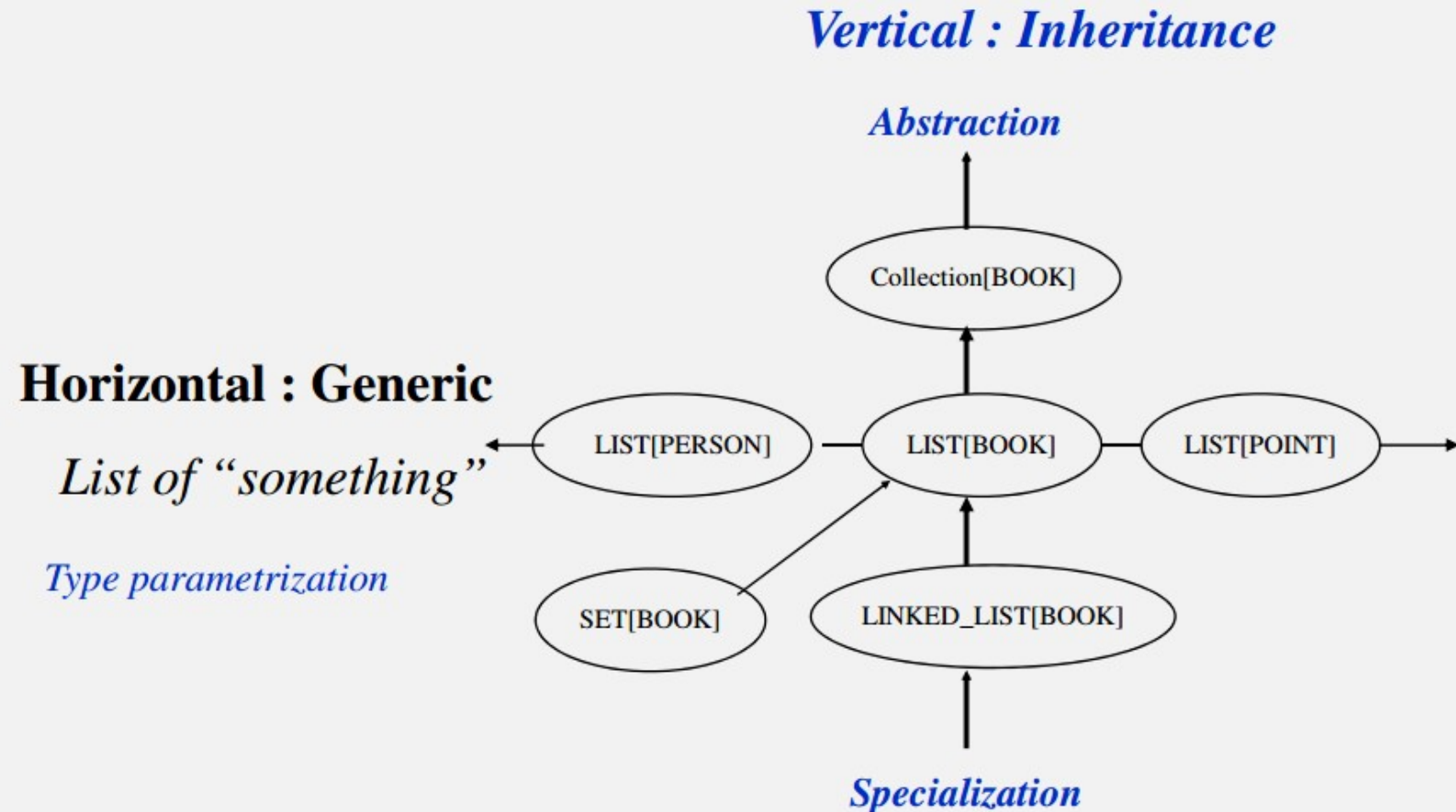

Kelas Generik

Generic Class

- Kelas generic: kelas yang masih “umum”, belum spesifik ketika didefinisikan
- Pada saat deklarasi objek, hal yang umum harus dibuat spesifik
 - Setelah dibuat spesifik, baru bisa dipakai
- Biasanya yang “umum” adalah “type”-nya, dipakai untuk membungkus “operasi” yang sama
- Dalam bahasa C++ menjadi *template*

Generic vs Inheritance

- **Inheritance** (vertikal):
 - variasi perilaku
 - relasi is-a
 - dynamic binding
- **Template** (horizontal):
 - variasi tipe
 - relasi “of T”
 - compile-time instantiation



Konsep template pada kelas

- Kelas generik merupakan penerapan konsep template pada kelas.
- Dengan demikian cukup mendefinisikan satu kelas generik (misal kelas Stack generik) untuk dapat diinstansiasikan sebagai Stack of int, Stack of float, Stack of char, Stack of Complex, Stack of String, dst.
- Untuk menciptakan kelas generik, perancang kelas harus dapat mengidentifikasi parameter-parameter mana yang menentukan sifat kelas.
 - Parameter template menentukan tipe elemen yang akan digunakan oleh kelas, tanpa mengubah tanggung jawab kelas tersebut.
 - Pada contoh Stack yang menyimpan array bertipe int, int merupakan kandidat parameter template.

Contoh Kelas Generik

- Kelas Point yang masih generic:
 - `GPoint <Numeric>`: kelas Point dengan absis dan ordinat ber-type numerik
 - Saat dideklarasikan:
 - `GPoint <integer> P`; maka absis dan ordinat akan ber-type integer
 - `GPoint <float> P`; maka absis dan ordinat akan ber-type float
- Kelas `GArray <a_type>`: array of elemen bertipe `a_type`
 - Pada saat dideklarasikan dapat menjadi: array of `<integer>`, array of `<float>`, array of `<Point>`, ...

Contoh: kelas Stack generik

```
template <class T>
class Stack {
public:
    void push(const T& item) {
        data_.push_back(item);
    }

    T pop() {
        return data_.pop_back();
    }

    bool empty() const {
        return data_.empty();
    }

private:
    std::vector<T> data_;
};
```

Catatan-catatan

- Penciptaan objek dilakukan sebagai berikut:

```
Stack<int> a;           // Stack of integer  
Stack<double> b(30);   // Stack of double, kapasitas maks = 30  
Stack<Complex> c;      // Stack of Complex
```

- Nama Stack<int>, Stack<double>, ... dapat dipandang sebagai nama tipe baru.

Catatan-catatan

- Definisi fungsi anggota harus dituliskan sebagai fungsi template dan scope yang semula dituliskan sebagai `Stack::` harus dituliskan sebagai `Stack<T>::`.

```
/* konstruktor (jika perlu) */  
template <class T>  
Stack<T>::Stack() {  
    ...  
}
```

```
/* fungsi anggota */  
template <class T>  
void Stack<T>::push(T item) {  
    data_.push_back(item);  
}
```


Catatan-catatan

- Deklarasi kelas generik maupun definisi fungsi generik dituliskan dalam file header (.hpp).
 - Ada cara-cara lain, lihat:
 - <https://stackoverflow.com/questions/495021/why-can-templates-only-be-implemented-in-the-header-file>,
 - <https://isocpp.org/wiki/faq/templates#templates-defn-vs-decl>

Catatan-catatan

- Di luar konteks definisi kelas generik, nama tipe yang dapat digunakan (misalnya oleh fungsi, deklarasi variabel/objek, dsb.) adalah nama tipe hasil instansiasi, seperti `Stack<int>`, `Stack<double>`, atau `Stack<Complex>`.
 - Termasuk di dalam definisi fungsi, contoh:

```
template <class T>
void Stack<T>::reverse() {
    Stack<T> stemp; // objek lokal yang generik
    // ...algoritma dari reverse()...
}
```

Spesialisasi kelas generik

- Kita dapat mendefinisikan perilaku spesial untuk kelas generik yang memiliki tipe tertentu sebagai *template parameter*.
- *Template specialization* memilih implementasi berbeda saat *compile-time*, bukan melalui dynamic binding.

```
// class template:
template <class T>
class MyContainer {
    T element;
public:
    MyContainer(T arg) { element = arg; }
    T increase() { return ++element; }
};
```

```
// class template specialization:
template <>
class MyContainer<char> {
    char element;
public:
    MyContainer(char arg) { element = arg; }
    char uppercase() { // perilaku khusus untuk MyContainer <char>
        if ((element >= 'a') && (element <= 'z'))
            element += 'A' - 'a';
        return element;
    }
};
```

Takeaway

- *Template* memungkinkan:
 - Abstraksi generik atas tipe,
 - Implementasi algoritma dan struktur data yang sama untuk berbagai tipe,
 - *Reuse* tanpa membentuk relasi hierarki,
 - Resolusi pada *compile-time*, bukan *run-time*.
- *Template* melengkapi *inheritance*:
 - *Inheritance* → variasi perilaku (vertikal)
 - *Template* → variasi tipe (horizontal)
- Dengan demikian, template bukan mekanisme *polymorphism*, melainkan mekanisme parameterisasi tipe dalam sistem OO.